```
    "title" : "Effective Java",
    "edition" : 3,
    ...
  }
}]
```

If you look carefully at the result, the score is 1.0 by default because term-level queries are not concerned with relevancy, as I mentioned earlier.

2.4.2 The range query

A `range` query fetches results that match a range: for example, fetching flights between 1:00 a.m. and 1:00 p.m. or finding a list of teenagers between ages 14 and 19. The `range` query can be applied to dates, numerals, and other attributes, making it a powerful companion when searching for range data.

Continuing our book example, we can use a `range` query to fetch all books with an `amazon_rating` greater than or equal to 4.5 stars and less than or equal to 5 stars.

Listing 2.17 A `range` query to fetch books rating 4.5 to 5 stars

```
GET books/_search
{
  "query": {
    "range": {
      "amazon_rating": {
        "gte": 4.5,
        "lte": 5
      }
    }
  }
}
```

Range query declaration

Range to match

gte: greater than or equal to

lte: less than or equal to

This `range` query will fetch three books, because we have three books whose ratings are greater than or equal to 4.5 (I've omitted the output for brevity).

There are a handful of term-level queries, including `terms`, `IDs`, `exists`, `prefix`, and others. We go over them in more detail in chapters 8–10.

So far, we've gone through queries that may be helpful when fetching results based on basic criteria: match on a title, search for a word in multiple fields, find the top-rated sellers, and so on. But in reality, queries can be as complex as you can imagine: for example, fetching first edition books authored by Joshua with a rating greater than 4.5 and published after 2015. Fortunately, Elasticsearch has advanced query types that we can use for complex criteria searches in the form of *compound queries*. Let's look at an example of a compound query in the next section.

2.5 Compound queries

Compound queries in Elasticsearch provide a mechanism to create sophisticated search queries. They combine individual queries, called *leaf queries* (like the ones we've seen so far), to build powerful, robust queries that cater to complex scenarios.